Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3
По дисциплине «Современные платформы программирования»

Выполнил:
Студент 3 курса
Группы ПО-12
Авдей Д.Б.
Проверил:
Кулик А.Д.

Брест 2026

Цель: приобрести навыки применения паттернов проектирования при решении практических задач с использованием языка Python.

Вариант №1

Задание 1

Реализовать фрагмент программной системы, используя выбранный паттерн (Порождающий). Реализовать все необходимые дополнительные классы. Кофе-автомат с возможностью создания различных кофейных напитков (предусмотреть 5 классов наименований).

Результат:

```
===  КОФЕ-АВТОМАТ ===
1 — Эспрессо
2 — Американо
3 — Капучино
4 — Латте
5 — Мокко
0 — Выход
Выберите напиток: 1
Готовим эспрессо
```

```
===  КОФЕ-АВТОМАТ ===
1 — Эспрессо
2 — Американо
3 — Капучино
4 — Латте
5 — Мокко
0 — Выход
Выберите напиток: 3
Готовим капучино
```

Код:

```
from abc import ABC, abstractmethod


class Coffee(ABC):
    @abstractmethod
    def prepare(self):
        pass


class Espresso(Coffee):
    def prepare(self):
        return "Готовим эспрессо"
```

```
class Americano(Coffee):
    def prepare(self):
        return "Готовим американо"


class Cappuccino(Coffee):
    def prepare(self):
        return "Готовим капучино"


class Latte(Coffee):
    def prepare(self):
        return "Готовим латте"


class Mocha(Coffee):
    def prepare(self):
        return "Готовим мокко"


class CoffeeMachine:
    def create_coffee(self, coffee_type: str) -> Coffee:
        if coffee_type == "1":
            return Espresso()
        if coffee_type == "2":
            return Americano()
        if coffee_type == "3":
            return Cappuccino()
        if coffee_type == "4":
            return Latte()
        if coffee_type == "5":
            return Mocha()

        raise ValueError("Неверный выбор напитка")

    def order_coffee(self, coffee_type: str):
        coffee = self.create_coffee(coffee_type)
        print(coffee.prepare())


def show_menu():
    print("\n===  КОФЕ-АВТОМАТ ===")
    print("1 — Эспрессо")
    print("2 — Американо")
    print("3 — Капучино")
    print("4 — Латте")
    print("5 — Мокко")
    print("0 — Выход")


def main():
    machine = CoffeeMachine()

    while True:
        show_menu()
        choice = input("Выберите напиток: ").strip()

        if choice == "0":
            print("До свидания!")
```

```
            break

        try:
            machine.order_coffee(choice)
        except ValueError as e:
            print(e)


if __name__ == "__main__":
    main()
```

Задание 2

Реализовать фрагмент программной системы, используя выбранный паттерн (Структурный). Реализовать все необходимые дополнительные классы. Проект «Часы». В проекте должен быть реализован класс, который дает возможность пользоваться часами со стрелками так же, как и цифровыми часами. В классе «Часы со стрелками» хранятся повороты стрелок.

Результат:

```
Введите угол часовой стрелки: 240
Введите угол минутной стрелки: 36
Текущее время: 08:06
```

```
Введите угол часовой стрелки: 120
Введите угол минутной стрелки: 66
Текущее время: 04:11
```

Код:

```
        from abc import ABC, abstractmethod


class DigitalClock(ABC):
    @abstractmethod
    def get_time(self) -> str:
        pass


class AnalogClock:
    def __init__(self, hour_angle: float, minute_angle: float):
        self._hour_angle = hour_angle % 720
        self._minute_angle = minute_angle % 360
```

```
    @property
    def hour_angle(self) -> float:
        return self._hour_angle

    @property
    def minute_angle(self) -> float:
        return self._minute_angle


class ClockAdapter(DigitalClock):
    def __init__(self, analog_clock: AnalogClock):
        self.analog_clock = analog_clock

    def get_time(self) -> str:
        ha = self.analog_clock.hour_angle
        ma = self.analog_clock.minute_angle

        minutes = int(ma / 6)
        hours = int(ha / 30)

        hours = hours % 24
        minutes = min(minutes, 59)

        return f"{hours:02d}:{minutes:02d}"


if __name__ == "__main__":
    input_hour_angle = float(input("Введите угол часовой стрелки: "))
    input_minute_angle = float(input("Введите угол минутной стрелки: "))

    analog = AnalogClock(input_hour_angle, input_minute_angle)
    digital_clock = ClockAdapter(analog)

    print("Текущее время:", digital_clock.get_time())
```

Задание 3

Реализовать фрагмент программной системы, используя выбранный паттерн (Поведенческий). Реализовать все необходимые дополнительные классы. Проект «Клавиатура настраиваемого калькулятора». Цифровые и арифметические кнопки имеют фиксированную функцию, а остальные могут менять своё назначение.

Результат:

```
Добавляем 5: 5
Вычитаем 2: 3
Умножаем на 10: 30
Делим на 3: 10.0
```

```python
calc = Calculator()

add5 = Button(AddCommand(calc, num: 5))
sub2 = Button(SubtractCommand(calc, num: 2))
reset = Button(ResetCommand(calc))

print("Добавляем 5:", add5.press())
print("Вычитаем 2:", sub2.press())

custom = Button()
custom.set_command(MultiplyCommand(calc, num: 10))
print("Умножаем на 10:", custom.press())

custom.set_command(DivideCommand(calc, num: 3))
print("Делим на 3:", custom.press())
```

Код:

```python
from abc import ABC, abstractmethod


class Calculator:
    def __init__(self):
        self.value = 0

    def add(self, num):
        self.value += num
        return self.value

    def subtract(self, num):
        self.value -= num
        return self.value

    def multiply(self, num):
        self.value *= num
        return self.value

    def divide(self, num):
```

```
        if num == 0:
            raise ValueError("Деление на ноль!")
        self.value /= num
        return self.value

    def reset(self):
        self.value = 0
        return self.value


class Command(ABC):
    @abstractmethod
    def execute(self):
        pass


class AddCommand(Command):
    def __init__(self, calculator, num):
        self.calculator = calculator
        self.num = num

    def execute(self):
        return self.calculator.add(self.num)


class SubtractCommand(Command):
    def __init__(self, calculator, num):
        self.calculator = calculator
        self.num = num

    def execute(self):
        return self.calculator.subtract(self.num)


class MultiplyCommand(Command):
    def __init__(self, calculator, num):
        self.calculator = calculator
        self.num = num

    def execute(self):
        return self.calculator.multiply(self.num)


class DivideCommand(Command):
    def __init__(self, calculator, num):
        self.calculator = calculator
        self.num = num

    def execute(self):
        return self.calculator.divide(self.num)


class ResetCommand(Command):
    def __init__(self, calculator):
        self.calculator = calculator

    def execute(self):
        return self.calculator.reset()


class Button:
```

```
    def __init__(self, command=None):
        self.command = command

    def set_command(self, command):
        self.command = command

    def press(self):
        if self.command:
            return self.command.execute()
        raise ValueError("Команда не назначена")


if __name__ == "__main__":
    calc = Calculator()

    add5 = Button(AddCommand(calc, 5))
    sub2 = Button(SubtractCommand(calc, 2))
    reset = Button(ResetCommand(calc))

    print("Добавляем 5:", add5.press())
    print("Вычитаем 2:", sub2.press())

    custom = Button()
    custom.set_command(MultiplyCommand(calc, 10))
    print("Умножаем на 10:", custom.press())

    custom.set_command(DivideCommand(calc, 3))
    print("Делим на 3:", custom.press())
```

Вывод: приобрел навыки применения паттернов проектирования при решении практических задач с использованием языка Python